

Dynamic Programming (II)

- Longest common subsequence
- From Trace Back to Back Tracking
 - Eight Queens

Yanlin Zhang | DSAA 2043 Spring 2026

Sometimes trace-back branches

		Y			
		B	A	C	
		0	0	0	0
A	0	0	1	1	
B	0	1	1	1	
C	0	1	1	2	

At C[2,2], both up and left still preserve the optimal value 1.

$X = ABC, Y = BAC$

AC

X	A	B	C
Y	B	A	C

BC

X	A	B	C
Y	B	A	C

Both are optimal LCSs of length 2. The previous slide's single walk no longer tells the whole story.

A single walk is not enough

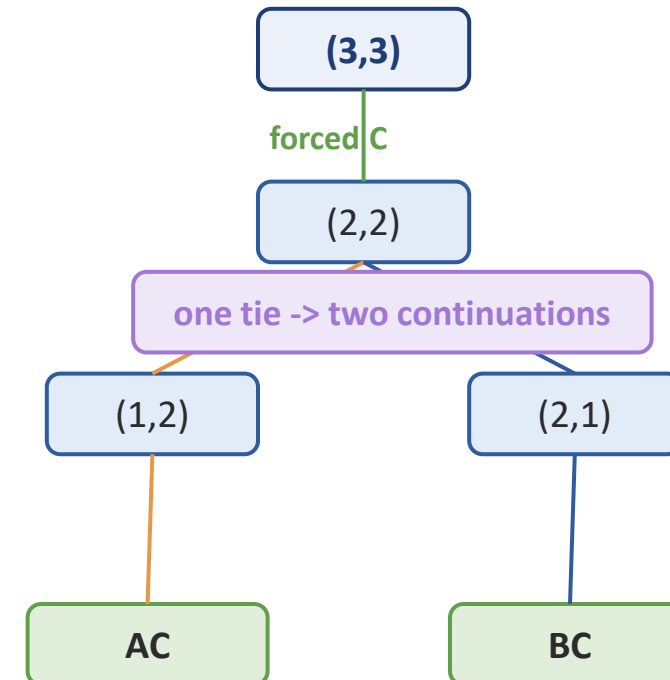
What changes at a tie?

match $\rightarrow$ go diagonal and append the symbol

strictly better side $\rightarrow$ follow it

tie $\rightarrow$ now there are two equally good next states

To list every optimal LCS, we must explore both branches.



What is Backtracking?

Formal definition

Backtracking is depth-first search on a tree of partial solutions.

We try one choice, recurse, and if that branch fails or finishes, we return to the previous choice point and try another choice.

Key ingredients: partial solution, choice point, recurse, return.

The return step is the “back-track” part.

choose



explore



return

then try others

Backtracking in LCS and 8-Queens

LCS: enumerate all optimal answers

state = current DP cell (i, j)

candidates = optimal predecessor cells

prune = discard moves that lower $C[i, j]$

output = every optimal LCS

8-Queens: enumerate all valid boards

state = current row + queens placed so far

candidates = safe columns in that row

prune = discard attacked positions

output = every legal placement

Same search pattern, different pruning rule.

What have we learned?

A single trace-back recovers one optimal LCS.

When ties appear, the trace-back becomes a search tree.

Backtracking = choose, recurse, return, then try other branches.

Next: 8-Queens uses the same search idea without a DP table.

Eight Queens Problem

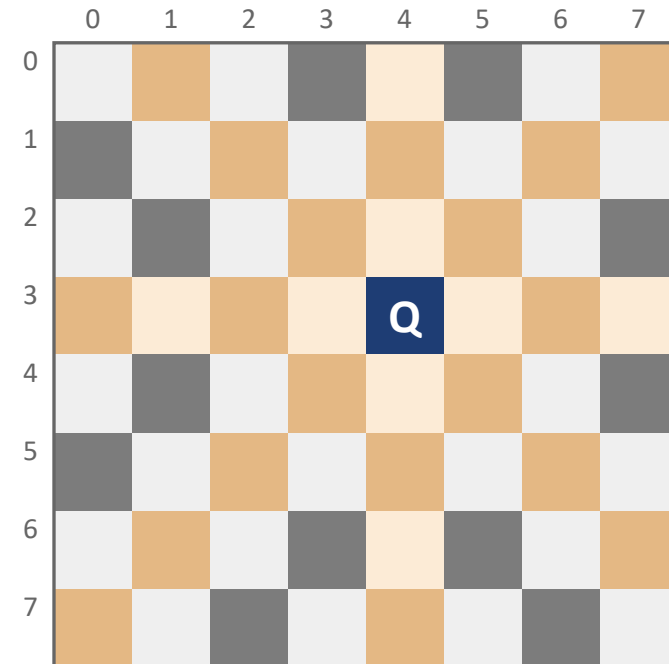
Place 8 queens on an 8 x 8 chessboard.

No two queens may share a row, a column, or a diagonal.

Equivalent search state: choose one column for each row.

state[row] = chosen column

Once the row is fixed, only the column is a decision.



One queen attacks along its row, column, and both diagonals.

Example Solution

	0	1	2	3	4	5	6	7
0	Q							
1					Q			
2								Q
3						Q		
4			Q					
5							Q	
6		Q						
7				Q				

One valid placement

$Q = [0, 4, 7, 5, 2, 6, 1, 3]$

Entry i stores the column chosen for row i .

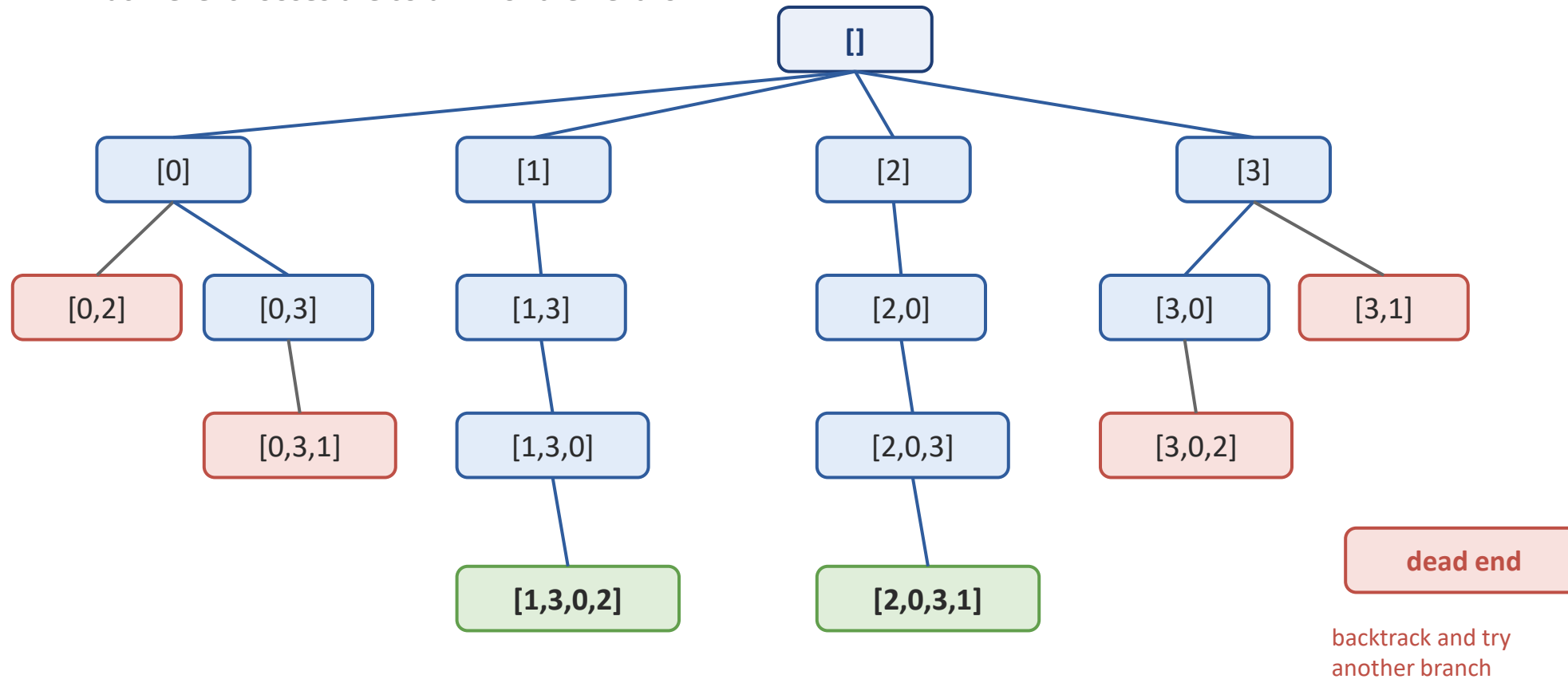
Every row is filled exactly once.

Backtracking builds this array from left to right.

Backtracking Search Tree

illustrated for $n = 4$

Each level chooses the column for the next row.



We place queens row by row, so recursion depth is the current row.

A node in the search tree is just a partial array `pos[0..row-1]`.

When testing (row, col), we only compare against earlier rows.

The moment a partial placement conflicts, we prune that branch.

Track three hash tables / sets

`usedCol[col]`

`usedDiag1[col - row]`

`usedDiag2[row + col]`

Keys are integers, so we do not need to shift indices anymore.

N-Queens Algorithm

```
Backtrack(row):  
    if row == n:  
        output solution  
        return  
  
    for col in 0 .. n-1:  
        if safe(row, col):  
            place queen  
            mark column and diagonals  
            Backtrack(row + 1)  
            unmark and remove queen
```

1. try a safe column

2. recurse to next row

3. undo on return

The undo step is what makes the search backtrack cleanly.

```
def solveNQueens(self, n):
    cols = set()
    diag1 = set()
    diag2 = set()
    board = [["."] * n for _ in range(n)]
    res = []
    def backtrack(row):
        if row == n:
            res.append(["".join(r) for r in board])
            return
        for col in range(n):
            if col in cols or (col - row) in diag1 or (row + col) in diag2:
                continue
            board[row][col] = "Q"
            cols.add(col)
            diag1.add(col - row)
            diag2.add(row + col)

            backtrack(row + 1)

            board[row][col] = "."
            cols.remove(col)
            diag1.remove(col - row)
            diag2.remove(row + col)
    backtrack(0)
    return res
```

Checking Safety in $O(1)$

No board scan is needed if we maintain three dicts/sets.

Each test becomes a few $O(1)$ hash

`usedCol[col]`

`usedDiag1[col - row]`

`usedDiag2[row + col]`

Example on an 8 x 8 board

try (row, col) = (3, 1)

diag1 key = $1 - 3 = -2$

diag2 key = $3 + 1 = 4$

With a dict/set, negative keys are fine.

Worst-case search is still exponential, roughly on the order of $n!$ nodes.

Backtracking cuts off bad partial placements immediately.

Space is $O(n)$: recursion depth n plus three bookkeeping arrays.

**Time
exponential**

**Space
 $O(n)$**

Stopping after the first solution is typically much faster than listing every solution.